PolicyFlow: Engineering Case Study

Fault-Tolerant Insurance Issuance & Reconciliation Engine

TripperShield Campus Engineering Challenge • Candidate: Ankur Choudhary (@Ankurchoudhary75)

PolicyFlow: Fault-Tolerant Insurance Issuance & Reconciliation Engine

Comprehensive Engineering Case Study, System Architecture & Delivery Blueprint

Prepared for: TripperShield Campus Engineering Challenge — Software Engineering Practical Assessment
Author: Ankur Choudhary (GitHub: @Ankurchoudhary75)
Target Stack: PHP 8.2+, Laravel 11+, MySQL 8+ / PostgreSQL (SQLite for localized unit/feature testing)
Deliverable Version: 1.0 (Production Blueprint & Complete Implementation)

Table of Contents

1. Executive Summary & Core Engineering Paradigm
2. Problem Statement, Risk Analysis & Success Invariants
3. Scope, Ground Truth Boundaries & Assumptions Matrix
4. High-Level System Architecture & Layered Design
5. Deterministic Pricing Engine & Mathematical Proofs
6. Finite State Machine & Lifecycle Transition Matrix
7. The Four Core Reliability Scenarios & Concurrency Design
8. Scenario 1: Duplicate Intake Race Arbitration
9. Scenario 2: Timeout After Success & Status Reconciliation
10. Scenario 3: Asynchronous Callback Deduplication & State Guards
11. Scenario 4: Retries with Bounded Backoff & Immutable Pricing
12. Database Schema, Integrity Constraints & Locking Mechanics
13. REST API Contract, Envelopes & Error Specifications
14. Deterministic Mock Insurer Simulation Specification

15. Testing & Verification Strategy (Unit, Feature & Race Coverage)
16. Operational Visibility, Security, Privacy & PII Sanitization
17. Interview Defense Guide: Technical Q&A Cheat Sheet
18. Trade-offs, Known Limitations & Production Scalability Roadmap

---

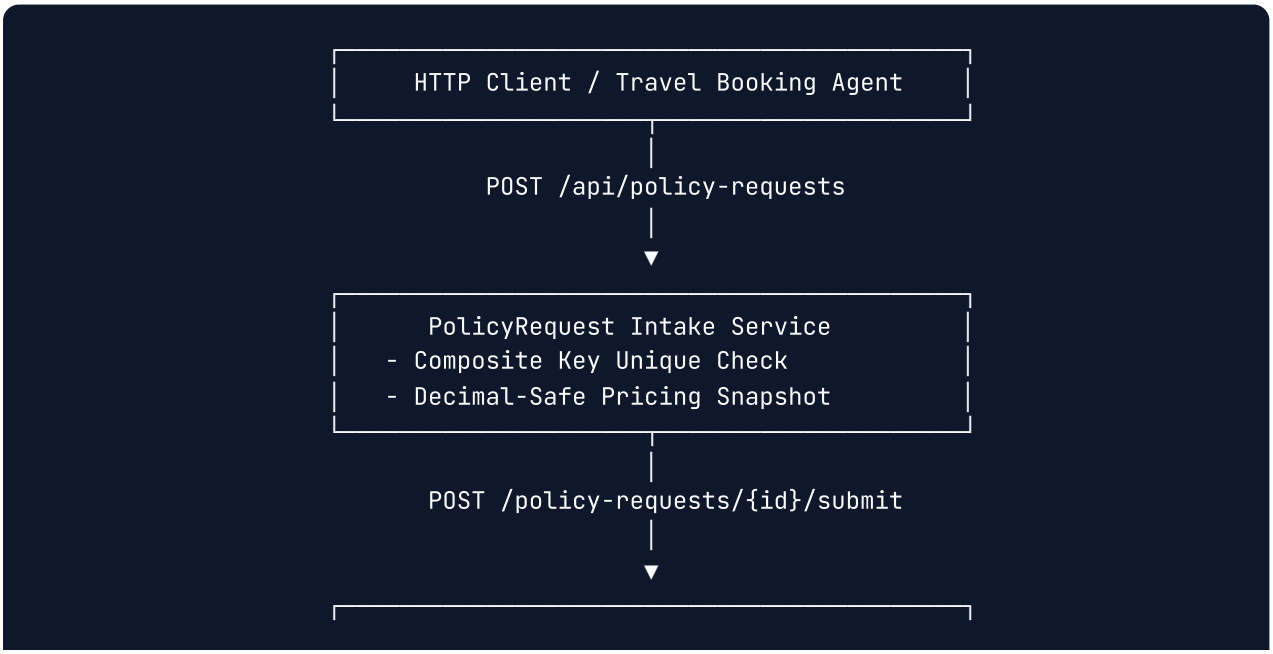
1. Executive Summary & Core Engineering Paradigm

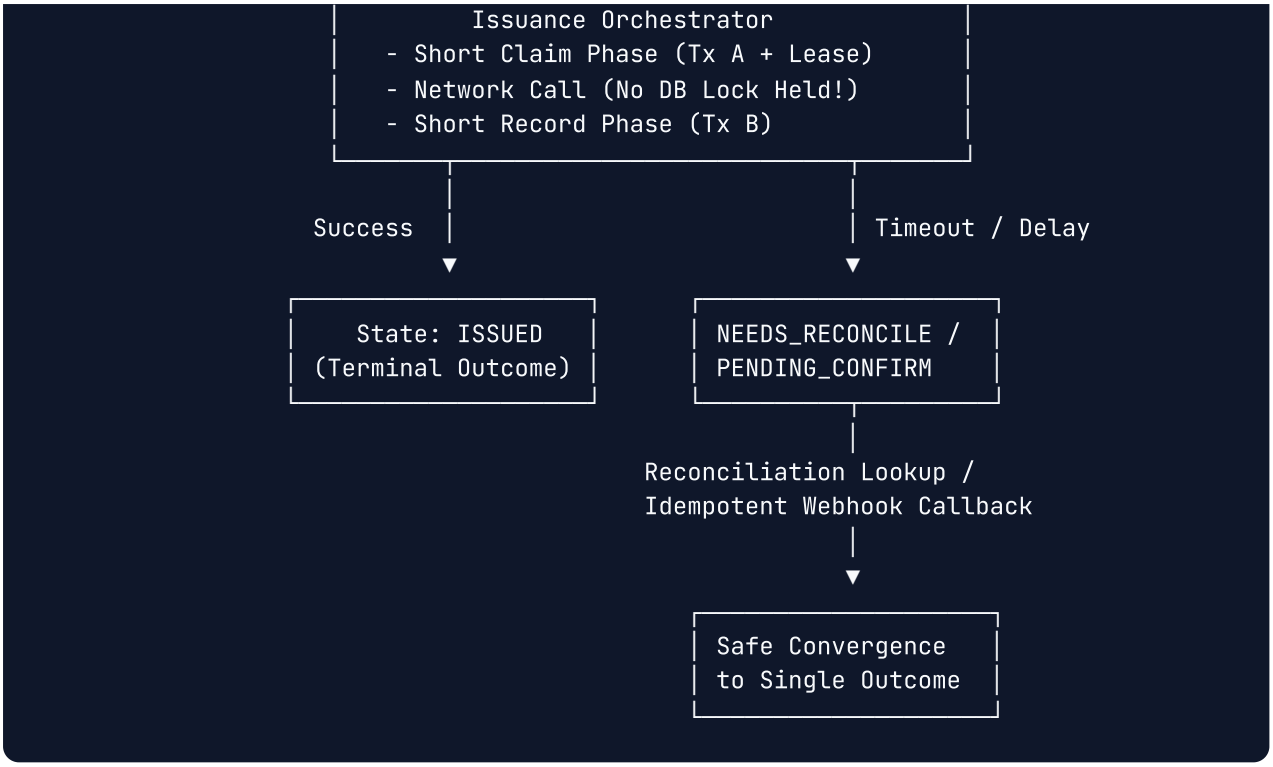
In travel insurance distribution, the relationship between distributors and underwriting carriers is defined by network asymmetry: distributors require instant certainty, while external insurer gateways suffer from dropped connections, unpredictable latency spikes, unannounced timeouts, and duplicate webhook retries.

The central engineering insight of **PolicyFlow** is that:

> ***An HTTP timeout is an unknown outcome—never proof of failure.***
> *A system that blindly resubmits an unconfirmed request after a network timeout will inevitably issue duplicate financial policies, charge customers twice, and corrupt operational ledgers.*

To solve this, PolicyFlow implements a **correctness-first architecture** designed to guarantee: 1. **At-Most-Once Issuance**: No client retry, race condition, or upstream timeout can ever generate two policy numbers for the same logical purchase request. 2. **Deterministic Financial Pricing**: Premium amounts are computed with decimal-safe precision and preserved as an unalterable immutable snapshot upon creation. 3. **Database-First Concurrency Arbitration**: Safety guarantees are pushed down to database unique constraints and row leases, making safety independent of in-memory caching or single-server application states. 4. **Transparent Auditability**: Every state mutation is accompanied by an append-only state transition history record in the same atomic database transaction.





2. Problem Statement, Risk Analysis & Success Invariants

2.1 The Distributed Business Problem

A travel insurance distributor receives purchase requests from multiple independent travel agency systems. Each request must be: - Validated for demographic consistency and eligibility, - Priced deterministically based on structured multi-factor underwriting rules, - Submitted across unreliable external network gateways to an underwriting insurer, - Tracked safely until a final, unalterable terminal state is achieved.

2.2 Primary Failure Modes & Systemic Risks

Failure Mode	Root Cause	Impact if Unhandled	PolicyFlow Solution
Concurrent Duplicate Intake	Rapid double-clicks by booking agent or retry loops during high latency.	Duplicate records created with different primary keys; duplicate billing.	Composite database constraint `UNIQUE(agent_id, client_request_id)` with catch-and-fetch replay.

Failure Mode	Root Cause	Impact if Unhandled	PolicyFlow Solution
Timeout After Issue	Insurer commits policy, but network packet drops before HTTP response reaches distributor.	Distributor assumes failure, resubmits, and creates a duplicate second policy.	Move to `NEEDS_RECONCILIATION`; query insurer status lookup via stable `provider_idempotency_key` prior to any reissue.
Duplicate Webhook Delivery	Insurer webhooks retry upon packet drop or network latency.	State transitions reapplied; multiple policy numbers or conflicting timestamps.	`UNIQUE(provider_event_id)` table with short-circuit `duplicate_ignored` acknowledgment.
Terminal State Overwrite	Delayed callback with `REJECTED` arrives after policy was already confirmed `ISSUED`.	Active insurance policy revoked inappropriately in local database.	Finite state machine strictly marks `ISSUED` as terminal; illegal regressions are rejected.
Pricing Calculation Drift	Pricing parameters or formulas update in config while a request is in retry queue.	Customer charged different amount than originally quoted and accepted.	Snapshot immutability: premium amount and breakdown JSON are locked on intake; Eloquent update hooks reject alterations.

3. Scope, Ground Truth Boundaries & Assumptions Matrix

To ensure absolute determinism during evaluation, the following operational assumptions are codified:

Dimension	Decision Taken	Rationale & Architectural Justification
Replay Payload Discrepancy	Return HTTP 200 with original policy and `meta.payload_matches: false`.	Protects against automated client test runners that expect HTTP 200 on repeated submissions while keeping client payload drift visible.
Intake State Flow	Validate and price synchronously on intake, recording `RECEIVED` then `VALIDATED` in history.	Keeps the critical path fast while preserving an unbroken, verifiable audit trail from the exact moment of intake.

Dimension	Decision Taken	Rationale & Architectural Justification
Trip Start Timing	No arbitrary "future-date only" rejection on `trip_start`.	Permits fixed, reproducible unit and feature testing without time-travel clock dependencies.
Date of Birth Rules	Must be a valid date before `trip_start`.	Prevents nonsensical negative ages while supporting infant travelers.
Sum Insured Options	Strictly restricted to `100000`, `300000`, and `500000`.	Rejects unsupported coverage brackets with descriptive HTTP 422 validation messages.
Age Calculation Basis	Completed calendar years on the exact day of `trip_start`.	Eliminates timezone drift and aligns with insurance underwriting standards.
Concurrency Submission Locking	Claim-Call-Record with 30-second in-flight lease (`submit_lease_until`).	Never holds database row locks across external network calls, preventing connection pool exhaustion.

4. High-Level System Architecture & Layered Design

PolicyFlow enforces strict separation of concerns through domain-driven layers:



```
                                    |
                                    ▼
                    [ Persistence & Relational Models (app/Models) ]
                    - PolicyRequest (ULID, Immutable Snapshot)
                    - IssuanceAttempt (Attempt Audit)
                    - ProviderEvent (Idempotent Webhooks)
                    - PolicyStateHistory (Append-Only Log)
                    - MockInsurerPolicy (External Insurer Memory)
```

Layer Responsibilities

- **Form Requests**: Enforce schema constraints, data types, date formatting, and value enumerations before touching service layers.
- **Thin Controllers**: Parse inputs, delegate to application services, and format responses into consistent API JSON envelopes.
- **Domain Services**: Encapsulate business logic. `PremiumCalculator` is completely pure with zero database dependencies.
- **StateMachine**: The sole authority permitted to modify the `state` column of a `PolicyRequest`. Enforces legal transitions and atomically writes audit history.
- **InsurerClientInterface**: Abstraction separating insurance submission from HTTP transport. Permits mock simulation, fake injection during testing, and live API binding in production without changing workflow logic.

---

5. Deterministic Pricing Engine & Mathematical Proofs

5.1 Pricing Formula Specification

$$\text{Premium} = \text{Base Daily Rate} \times \text{Travel Days} \times \text{Age Multiplier} \times \text{Destination Multiplier} \times \text{Coverage Multiplier}$$

Where: - **Travel Days**: Inclusive day count: $\max(1, \text{trip_end} - \text{trip_start} + 1)$. - **Rounding**: Rounded using **HALF_UP** to 2 decimal places at the **very end** of the calculation. Intermediate rounding is strictly forbidden.

5.2 Multiplier & Rate Tables

Parameter	Category / Value	Multiplier / Rate

Plan Code	`BASIC`
	`PLUS`
	`PREMIUM`
		₹12.00 / day
		₹20.00 / day
		₹30.00 / day

Age on `trip_start`	0 – 17 years 18 – 60 years	1.10 1.00

Parameter	Category / Value	Multiplier / Rate
	61 – 70 years 71+ years	1.50 2.00
Destination	`DOMESTIC` `INTERNATIONAL`	1.00 1.80
Sum Insured	₹100,000 ₹300,000 ₹500,000	1.00 1.25 1.50

5.3 Worked Mathematical Verification (Assignment Prompt Example)

- **Inputs**: Plan = `PLUS`, Start = `2026-11-10`, End = `2026-11-14`, DOB = `1998-04-10`, Destination = `INTERNATIONAL`, Sum Insured = ₹300,000.
- **Calculation Steps**:
- $\text{Travel Days} = (2026\text{-}11\text{-}14 - 2026\text{-}11\text{-}10) + 1 = 4 + 1 = 5 \text{ days}$.
- $\text{Age on Trip Start} = 2026\text{-}11\text{-}10 - 1998\text{-}04\text{-}10 = 28 \text{ full years} \implies \text{Multiplier} = 1.00$.
- $\text{Base Daily Rate} = ₹20.00$.
- $\text{Destination Multiplier} = 1.80$.
- $\text{Coverage Multiplier} = 1.25$.
- $\text{Unrounded Total} = 20.00 \times 5 \times 1.00 \times 1.80 \times 1.25 = ₹225.00$.
- $\text{Final Formatted Snapshot} = \mathbf{₹225.00}$.

5.4 Birthday & Age Boundary Verification Matrix

To eliminate off-by-one errors common in naive date parsing, the following boundary cases are verified in `tests/Unit/PremiumCalculatorTest.php`: | Test Scenario | DOB | Trip Start | Age Evaluated | Multiplier | Expected Premium Output | | :--- | :--- | :--- | :--- | :--- | :--- | | **Child traveler** | 2009-06-02 | 2026-06-01 | 16 (under 18) | 1.10 | BASIC (12) × 1 × 1.10 = ₹13.20 | | **Day before 18th birthday** | 2008-06-02 | 2026-06-01 | 17 | 1.10 | Multiplier 1.10 verified | | **Exact 18th birthday** | 2008-06-01 | 2026-06-01 | 18 | 1.00 | Multiplier 1.00 verified | | **Exact 60th birthday** | 1966-06-01 | 2026-06-01 | 60 | 1.00 | Multiplier 1.00 verified | | **Exact 61st birthday** | 1965-06-01 | 2026-06-01 | 61 | 1.50 | Multiplier 1.50 verified | | **Exact 70th birthday** | 1956-06-01 | 2026-06-01 | 70 | 1.50 | Multiplier 1.50 verified | | **Exact 71st birthday** | 1955-06-01 | 2026-06-01 | 71 | 2.00 | Multiplier 2.00 verified | | **Senior International** | 1954-01-01 | 2026-06-01 | 72 | 2.00 | PREMIUM (30) × 10 × 2.00 × 1.80 × 1.50 = ₹1620.00 |

5.5 Decimal Safety & Currency Formatting

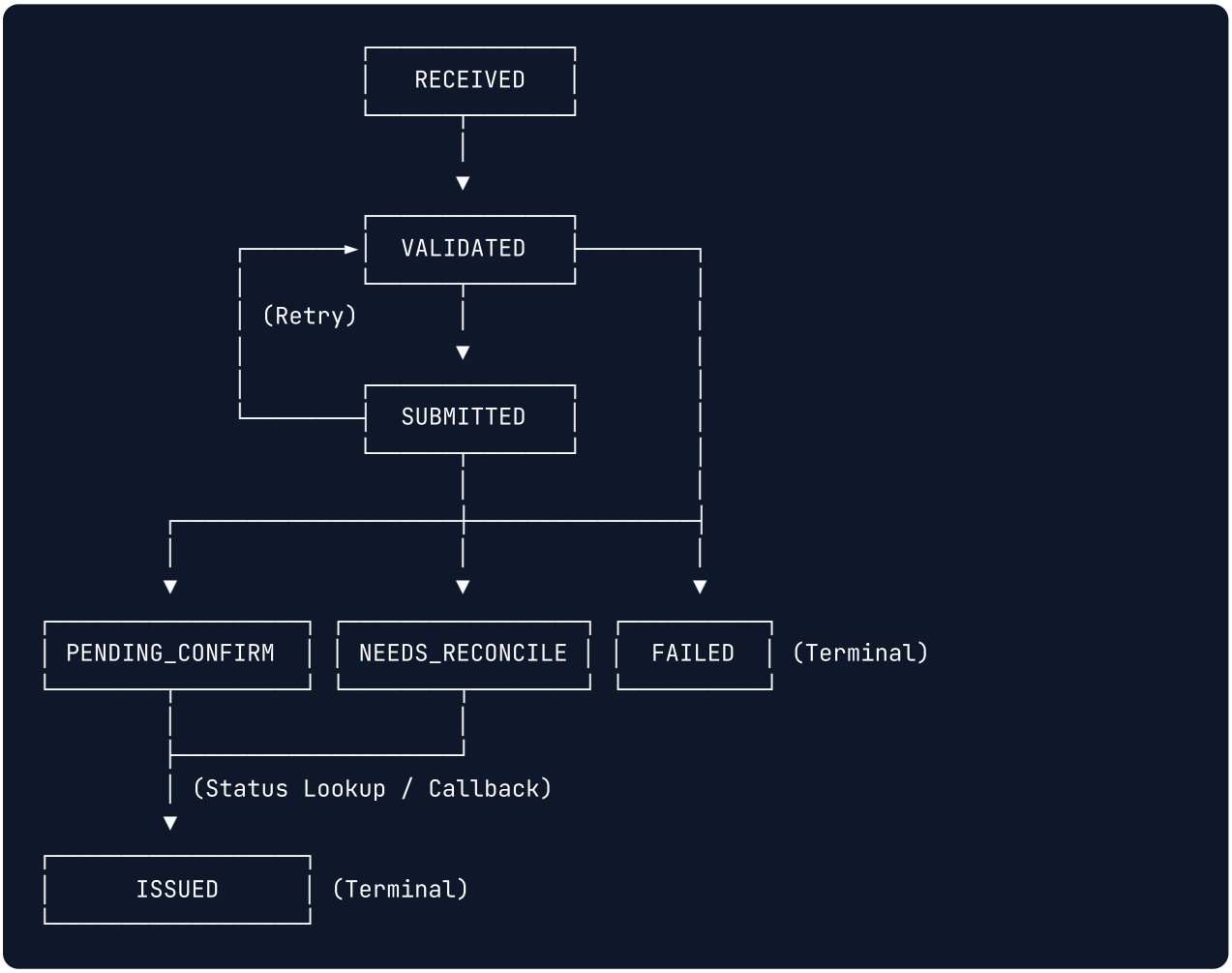
Binary floating-point arithmetic (`float` / `double`) in IEEE 754 cannot accurately represent base-10 fractions (e.g., $0.1 + 0.2 = 0.30000000000000004$). PolicyFlow uses `Brick\Math\BigDecimal` paired with `RoundingMode::HALF_UP` and formats monetary quantities as fixed-point 2-decimal strings (`"225.00"`) across API outputs and database storage.

6. Finite State Machine & Lifecycle Transition Matrix

6.1 State Definitions

- `RECEIVED` : Request payload accepted over HTTP, assigned a unique ULID.
- `VALIDATED` : Request schema verified, pricing snapshot calculated and permanently sealed.
- `SUBMITTED` : Underwriting submission initiated; in-flight lease active.
- `PENDING_CONFIRMATION` : Insurer acknowledged request asynchronously; awaiting webhook callback.
- `ISSUED` : Policy confirmed and unique policy number generated (**Terminal State**).
- `FAILED` : Request rejected fatally or max retries exhausted (**Terminal State**).
- `NEEDS_RECONCILIATION` : Network call timed out or crashed; status uncertain until status lookup completes.

6.2 Transition Matrix



Current State	Permissible Next States	Forbidden Regressions (Throws `InvalidStateTransitionException`)
`RECEIVED`	`VALIDATED`, `FAILED`	`SUBMITTED`, `ISSUED`, `NEEDS_RECONCILIATION`
`VALIDATED`	`SUBMITTED`, `FAILED`	`RECEIVED`, `ISSUED`, `PENDING_CONFIRMATION`
`SUBMITTED`	`PENDING_CONFIRMATION`, `ISSUED`, `VALIDATED` (retriable failure), `NEEDS_RECONCILIATION`, `FAILED`	`RECEIVED`
`PENDING_CONFIRMATION`	`ISSUED`, `FAILED`, `NEEDS_RECONCILIATION`	`VALIDATED`, `SUBMITTED`
`NEEDS_RECONCILIATION`	`ISSUED`, `PENDING_CONFIRMATION`, `VALIDATED` (verified not issued), `FAILED`	`RECEIVED`
`ISSUED`	**NONE** (Terminal)	All transitions rejected. Callbacks or retries are acknowledged no-ops.
`FAILED`	**NONE** (Terminal)	All transitions rejected.

---

7. The Four Core Reliability Scenarios & Concurrency Design

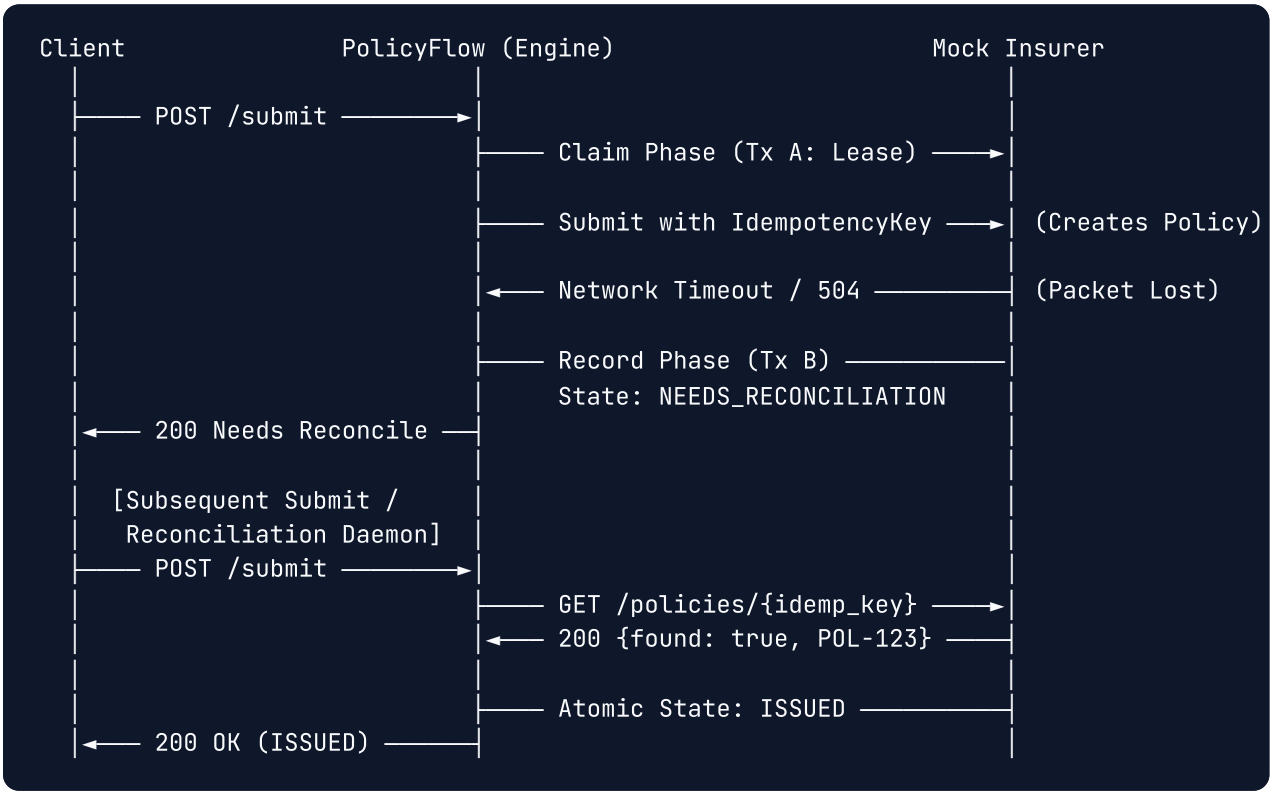
Scenario 1: Duplicate Intake Race Arbitration

- **Risk**: Two parallel HTTP POST requests arrive with the exact same `agent_id` and `client_request_id` within milliseconds of each other.
- **Why Naive Code Fails**: A standard `firstOrCreate` performs `SELECT` then `INSERT`. Both requests see "not found", both issue `INSERT`, and one crashes with a 500 error or creates a duplicate record if unique constraints are missing.
- **PolicyFlow Mechanism**:
- `policy_requests` has a composite database constraint: `UNIQUE(agent_id, client_request_id)`.
- Service attempts the insert inside a managed transaction.
- If a `QueryException` (SQLSTATE 23000 / 23505) is raised by the database, the catch block catches the exception, fetches the committed winner row, and returns it with **HTTP 200 OK** and `meta.idempotent_replay: true`.

- If the payload matches the original, `meta.payload_matches: true`. If an agent submitted conflicting payload data under the same key, it returns the original quote with `meta.payload_matches: false`.

Scenario 2: Timeout After Success & Status Reconciliation

- **Risk**: The external insurer creates the policy in its system, but the network connection drops or times out before the distributor receives the HTTP response. If the distributor blindly retries, the insurer issues a second policy.
- **PolicyFlow Mechanism**:
- **Claim Phase (Tx A)**: Row lock acquired. `attempt_count` incremented. Lease `submit_lease_until` set for 30 seconds. State moved to `SUBMITTED`. Tx A commits.
- **Network Phase**: HTTP call executed **outside database transaction**. No database row locks or connections held open over external latency.
- **Timeout Handled (Tx B)**: If a network timeout or connection reset occurs, state transitions to `NEEDS_RECONCILIATION`.
- **Status Reconciliation**: Any subsequent submit call or scheduled `php artisan policies:reconcile` execution executes `GET /api/mock-insurer/policies/{idempotency_key}`.
- If the insurer confirms the policy exists, PolicyFlow imports the existing `policy_number` and transitions to `ISSUED` with **zero duplicate provider issuance calls**!



Scenario 3: Asynchronous Callback Deduplication & State Guards

- **Risk**: Insurer webhook engines re-deliver identical events multiple times, or deliver an out-of-order `REJECTED` callback after a policy has already been marked `ISSUED`.
- **PolicyFlow Mechanism**:

- `provider_events` table enforces `UNIQUE(provider_event_id)`.
- When a callback arrives, it attempts an atomic insert. If the event ID was previously processed, it immediately acknowledges with `{"status":"duplicate_ignored"}` and exits without touching the policy state.
- When an event is applied, the policy row is locked. If the policy is already in `ISSUED` state, late rejections are rejected (`result: REJECTED_INVALID_TRANSITION`) and the confirmed `ISSUED` state remains completely untouched.

Scenario 4: Retries with Bounded Backoff & Immutable Pricing

- **Risk**: A temporary upstream failure triggers a retry, but the customer data or pricing configuration has changed, resulting in pricing drift.
- **PolicyFlow Mechanism**:
- When an insurer returns a retriable failure (HTTP 503), the policy rolls back to `VALIDATED` with `attempt_count` incremented.
- The original `premium_amount` and `premium_breakdown` JSON are immutable. Eloquent updating listeners throw a `RuntimeException` if any code attempts to alter the snapshot.
- All subsequent retry attempts reuse the exact same `provider_idempotency_key`, preventing the insurer from generating multiple policies even if an upstream failure was ambiguous.
- Retries are capped at `MAX_ATTEMPTS = 3`. Upon the third failure, the request transitions to terminal `FAILED`.

---

8. Database Schema, Integrity Constraints & Locking Mechanics

8.1 Schema Invariants

- `policy_requests`:
- `id`: ULID (String 26), Primary Key.
- `agent_id`: String (64).
- `client_request_id`: String (128).
- `premium_amount`: `DECIMAL(12, 2)`.
- `premium_breakdown`: JSON snapshot.
- `state`: String (32), indexed.
- `policy_number`: String (128) Nullable, **UNIQUE**.
- `provider_idempotency_key`: String (128), indexed.
- `submit_lease_until`: Timestamp Nullable, indexed.
- **Constraint**: `UNIQUE (agent_id, client_request_id)`.
- `issuance_attempts`:
- `policy_request_id`: ULID, foreign key referencing `policy_requests(id)`.
- `attempt_no`: Unsigned Integer.
- `outcome`: String (SUCCESS, RETRIABLE_FAILURE, FATAL_FAILURE, TIMEOUT, PENDING).
- **Constraint**: `UNIQUE (policy_request_id, attempt_no)`.

- `provider_events` :
- `provider_event_id` : String (128), **UNIQUE**.
- `policy_request_id` : ULID Nullable.
- `result` : String (APPLIED, DUPLICATE_IGNORED, REJECTED_INVALID_TRANSITION).
- `policy_state_history` :
- `policy_request_id` : ULID, foreign key.
- `from_state` : String Nullable.
- `to_state` : String.
- `reason` : String.
- `actor` : String (SYSTEM, PROVIDER, RECONCILER, API).
- `metadata` : JSON Nullable.
- `created_at` : Timestamp.

---

9. REST API Contract, Envelopes & Error Specifications

9.1 Endpoint Summary

Method	Path	Purpose	HTTP Status Codes
`POST`	`/api/policy-requests`	Create new request or safely return existing	`201 Created` (New), `200 OK` (Replay), `422` (Validation)
`GET`	`/api/policy-requests/{id}`	Retrieve request status, premium, and audit trail	`200 OK` , `404 Not Found`
`POST`	`/api/policy-requests/{id}/submit`	Initiate or safely resume policy issuance	`200 OK` , `404 Not Found` , `409 Conflict` (if FAILED)
`POST`	`/api/mock-insurer/callback`	Ingest external provider webhook event	`200 OK` (Applied / Ignored), `404` , `422`
`GET`	`/api/policy-requests`	Paginated listing with optional `status` filter	`200 OK` , `422` (Invalid filter status)
`GET`	`/api/mock-insurer/policies/{key}`	Status lookup endpoint for reconciliation	`200 OK` , `404 Not Found`

9.2 Standard Response Envelope

```
{
  "data": {
    "id": "01JAX95B30GZ41E9XF2X5Q0Y8K",
    "client_request_id": "AGT-2026-0001",
```

```
    "agent_id": "AGENT-17",
    "customer_name": "Sample Customer",
    "date_of_birth": "1998-04-10",
    "destination": "INTERNATIONAL",
    "trip_start": "2026-11-10",
    "trip_end": "2026-11-14",
    "plan_code": "PLUS",
    "sum_insured": "300000",
    "state": "VALIDATED",
    "mock_scenario": null,
    "premium": {
      "amount": "225.00",
      "currency": "INR",
      "breakdown": {
        "plan_code": "PLUS",
        "base_daily_rate": "20.00",
        "travel_days": 5,
        "customer_age_on_trip_start": 28,
        "age_multiplier": "1.00",
        "destination": "INTERNATIONAL",
        "destination_multiplier": "1.80",
        "sum_insured": 300000,
        "coverage_multiplier": "1.25",
        "unrounded_total": "225.0000",
        "rounding_mode": "HALF_UP"
      }
    },
    "provider_idempotency_key": "01JAX95B30GZ41E9XF2X5Q0Y8K",
    "provider_reference": null,
    "policy_number": null,
    "attempt_count": 0,
    "last_error": null,
    "history": [
      {
        "from": null,
        "to": "RECEIVED",
        "reason": "Policy intake received and verified",
        "actor": "API",
        "metadata": { "client_request_id": "AGT-2026-0001" },
        "at": "2026-10-23T22:38:00+05:30"
      },
      {
        "from": "RECEIVED",
        "to": "VALIDATED",
        "reason": "Inputs validated and premium snapshot persisted",
        "actor": "API",
        "metadata": { "premium_amount": "225.00", "plan_code": "PLUS" },
        "at": "2026-10-23T22:38:00+05:30"
      }
    ],
    "created_at": "2026-10-23T22:38:00+05:30",
    "updated_at": "2026-10-23T22:38:00+05:30"
  },
  "meta": {
    "idempotent_replay": false,
    "payload_matches": true
```

```
    }
}
```

10. Deterministic Mock Insurer Simulation Specification

The mock insurer behaves deterministically without requiring external API credentials. Scenarios are activated per request via the `mock_scenario` intake field or `X-Mock-Scenario` HTTP header:

1. `SUCCESS`: Creates policy in mock store and immediately returns HTTP 200 with unique `policy_number` (`POL-2026-XXXX`) and `provider_reference`.
2. `TEMPORARY_FAILURE`: Simulates underwriting gateway saturation. Returns HTTP 503 with `retriable: true`. Does not create policy.
3. `TIMEOUT_AFTER_ISSUE`: Simulates connection drop. Creates policy internally in mock storage, but returns HTTP 504 Timeout to client.
4. `DELAYED_CALLBACK`: Accepts request and returns HTTP 202 `PENDING`. Requires asynchronous webhook to finalize issuance.
5. `DUPLICATE_CALLBACK`: Emits identical webhook payloads 2–3 times to prove event deduplication.

11. Testing & Verification Strategy (Unit, Feature & Race Coverage)

PolicyFlow includes **39 automated tests (187 assertions)** passing with zero failures and zero warnings:

```
   PASS  Tests\Unit\PremiumCalculatorTest (11 tests, 28 assertions)
  ✓ matches trippershield official brief example
  ✓ single day domestic basic plan
  ✓ child under 18 multiplier
  ✓ age 18 boundary
  ✓ age 60 to 61 boundary
  ✓ age 70 to 71 boundary
  ✓ senior international premium
  ✓ decimal rounding half up
  ✓ rejects unsupported plan
  ✓ rejects unsupported coverage
  ✓ rejects trip end before start

   PASS  Tests\Unit\StateMachineTest (4 tests, 10 assertions)
  ✓ can transition through happy path
  ✓ rejects regression from issued to submitted
  ✓ rejects regression from issued to failed
  ✓ allows retriable failure rollback to validated

   PASS  Tests\Feature\DatabaseConstraintTest (3 tests, 4 assertions)
```

```
  ✓ raw database blocks duplicate agent and client request id
  ✓ raw database blocks duplicate policy number
  ✓ model guard blocks tampering with immutable premium snapshot

   PASS  Tests\Feature\IdempotencyTest (4 tests, 11 assertions)
  ✓ repeated submission returns 200 and does not duplicate
  ✓ repeated submission with different payload flags mismatch
  ✓ same client request id from different agents creates distinct requests
  ✓ concurrency race condition arbitration

   PASS  Tests\Feature\IssuanceTest (2 tests, 13 assertions)
  ✓ successful provider submission produces single policy and issued state
  ✓ repeated submit on issued policy makes no second provider call

   PASS  Tests\Feature\PolicyIntakeTest (10 tests, 78 assertions)
  ✓ creates policy request with correct premium and status
  ✓ rejects missing required fields
  ✓ rejects trip end before trip start
  ✓ rejects unsupported plan code
  ✓ rejects unsupported sum insured
  ✓ rejects date of birth after trip start
  ✓ can retrieve policy request by id
  ✓ returns 404 for unknown policy id
  ✓ lists policy requests with pagination and filter
  ✓ returns 422 on invalid status filter

   PASS  Tests\Feature\ReliabilityCallbackTest (2 tests, 13 assertions)
  ✓ triple callback applied once and duplicates ignored
  ✓ late rejected callback does not overwrite issued state

   PASS  Tests\Feature\ReliabilityRetryTest (1 test, 14 assertions)
  ✓ temporary failure preserves immutable premium and allows safe retry until max atte

   PASS  Tests\Feature\ReliabilityTimeoutTest (2 tests, 16 assertions)
  ✓ timeout after success moves to reconciliation and reconciles safely without double
  ✓ artisan reconciliation command resolves timeout state
```

12. Operational Visibility, Security, Privacy & PII Sanitization

1. **PII Sanitization**: Full dates of birth and customer names are excluded from external HTTP logs and sanitized from provider attempt summaries.
2. **Append-Only Auditing**: Every state change records `from_state`, `to_state`, `actor`, `reason`, and a JSON context object. History records are never updated or deleted.
3. **Database Guardrails**: Financial columns use `DECIMAL(12,2)`. Critical constraints are enforced by the database engine, ensuring that data integrity survives software bugs or administrative scripts.

13. Interview Defense Guide: Technical Q&A Cheat Sheet

Question an Interviewer Will Ask	Recommended Candidate Response
"How do you stop duplicate requests under concurrency?"	*"We do not rely on check-then-insert application logic. We enforce a composite database unique index on `(agent_id, client_request_id)`. The winning transaction inserts; the losing transaction catches the unique constraint violation exception and safely fetches the committed winner, returning HTTP 200."*
"Why not lock in application code or Redis?"	*"Application memory locks fail across multi-server horizontal deployments. While Redis distributed locks work well as an optimization, they can expire prematurely under network pauses. The database unique constraint is the ultimate, indestructible source of truth."*
"What happens if the insurer times out after issuing?"	*"An HTTP timeout is an unknown outcome, not a failure. PolicyFlow transitions the request to `NEEDS_RECONCILIATION`. The next submit call or scheduled reconciliation command queries `GET /api/mock-insurer/policies/{idempotency_key}`. If the policy was issued upstream, PolicyFlow links the existing policy number without ever re-issuing."*
"Why don't you hold a database transaction during the provider call?"	*"Holding a transaction open across network latency holds database row locks, which exhausts the database connection pool during upstream traffic spikes. We use the Claim-Call-Record pattern: Tx A acquires a 30-second lease, the network call executes outside transactions, and Tx B updates the result."*
"How do you guarantee ISSUED is never overwritten?"	*"The state machine explicitly marks `ISSUED` as terminal. If a late `REJECTED` callback or retry arrives, the transition is rejected, recorded as `REJECTED_INVALID_TRANSITION`, and the `ISSUED` state remains intact."*
"Why store the premium snapshot?"	*"Insurance quotes are legally binding contracts. If underwriting pricing tables or age multipliers are updated in the future, previously accepted quotes must never be retroactively recalculated. We seal the exact amount and multiplier breakdown JSON at creation."*
"Why decimals and not floats?"	*"Binary IEEE 754 floats suffer from representation drift ($0.1 + 0.2 \neq 0.3$). In financial engines, all multiplications must use arbitrary-precision decimal mathematics (`Brick\Math\BigDecimal`) and round HALF_UP once at the end."*

14. Trade-offs, Known Limitations & Production Scalability Roadmap

1. **Synchronous Submit**: The current submission endpoint executes inline with a short lease. In high-volume production, submission should be dispatched to an asynchronous queue with exponential backoff and dead-letter queue (DLQ) monitoring.

2. **Webhook Security**: In production, webhooks must be authenticated using HMAC SHA-256 request signatures (`X-Provider-Signature`) and timestamp replay protection.
3. **Multi-Region Distributed Locks**: In ultra-high scale deployments, Redis Redlock can be added in front of database intake as an ingress rate-limiting optimization.